Meeting 5 SP23

# TensorDIMM

Youngeun Kwon, Yunjae Lee, Minsoo Rhu, MICRO-52
presented by @crawfish, @timot

March 1, 2023

# Outline

Background

Motivation

TensorDIMM

Evaluation

Section 1

# Background

# DRAM Architecture

This is a DIMM (Dual In-Line Memory Module)

# DRAM Buffer

- Traditionally a single memory controller is responsible for interfacing with all DIMMs

# DRAM Buffer

- Traditionally a single memory controller is responsible for interfacing with all DIMMs
  - This is ineffective for server-class DRAM where the number of DIMMs can be large

# DRAM Buffer

- Traditionally a single memory controller is responsible for interfacing with all DIMMs
  - This is ineffective for server-class DRAM where the number of DIMMs can be large
  - Employ 'buffer' device per DIMM $\rightarrow$ intermediate controller

# DRAM Buffer

- Traditionally a single memory controller is responsible for interfacing with all DIMMs
  - This is ineffective for server-class DRAM where the number of DIMMs can be large
  - Employ 'buffer' device per DIMM $\rightarrow$ intermediate controller
- Prior work has shown this buffer device can be used for custom logic
  - IBM CentaurDIMM

# System Architecture for Deep Learning

- Growing trend towards dense, scaled-up system node design with multiple PCIe-attached co-processor devices

# System Architecture for Deep Learning

- Growing trend towards dense, scaled-up system node design with multiple PCIe-attached co-processor devices
- Inter-device communication lies on the critical path of DL applications

# System Architecture for Deep Learning

- Growing trend towards dense, scaled-up system node design with multiple PCIe-attached co-processor devices
- Inter-device communication lies on the critical path of DL applications
  - Vendors are employing high-bandwidth interconnection fabrics with custom signaling links

# System Architecture for Deep Learning

- Growing trend towards dense, scaled-up system node design with multiple PCIe-attached co-processor devices
- Inter-device communication lies on the critical path of DL applications
  - Vendors are employing high-bandwidth interconnection fabrics with custom signaling links
- **System Goal:** Faster data transfers

# Embeddings

*An embedding is a relatively low-dimensional space into which you can translate high-dimensional vectors*

- Embeddigns are trained to extract deep learning features s (e.g., Facebook trains embeddings to extract information regarding what pages a particular user liked)

# Embeddings

*An embedding is a relatively low-dimensional space into which you can translate high-dimensional vectors*

- Embeddigns are trained to extract deep learning features s (e.g., Facebook trains embeddings to extract information regarding what pages a particular user liked)
- Embeddings make it easier to do machine learning on large inputs like sparse vectors representing words

# Embeddings

*An embedding is a relatively low-dimensional space into which you can translate high-dimensional vectors*

- Embeddigns are trained to extract deep learning features s (e.g., Facebook trains embeddings to extract information regarding what pages a particular user liked)
- Embeddings make it easier to do machine learning on large inputs like sparse vectors representing words
- Embeddings are generally very sparse (information is not grouped together in memory).

# DL Applications with Embeddings

- DNN-based recommender systems
  - Problem: predict likelihood of certain event (e.g clicking into a YouTube video)

# DL Applications with Embeddings

- DNN-based recommender systems
  - Problem: predict likelihood of certain event (e.g clicking into a YouTube video)
- Embedding lookups and tensor manipulation
  - Inputs to DNNs are constructed as a combo of dense and sparse features
  - Sparse features are used to query embedding lookup tables
  - The output of the lookup tables are concatenated ("reduced) to form an output embedding, which is fed to the model.

# DL Applications with Embeddings

- DNN-based recommender systems
  - Problem: predict likelihood of certain event (e.g clicking into a YouTube video)
- Embedding lookups and tensor manipulation
  - Inputs to DNNs are constructed as a combo of dense and sparse features
  - Sparse features are used to query embedding lookup tables
  - The output of the lookup tables are concatenated ("reduced) to form an output embedding, which is fed to the model.
- Memory capacity limits of embedding layers
  - Embeddings scale with the number of items.
  - If a user is an item, a social media platform's embedding table can exceed several hundred GB

Section 2

# Motivation

# Memory Scaling Challenges

- GPUs often employ high-bandwidth memory (eg. GDDR, HBM)

# Memory Scaling Challenges

- GPUs often employ high-bandwidth memory (eg. GDDR, HBM)
- HBM is often capacity-limited when compared to CPU DRAM
- Technological constraints to adding more RAM:
  - Stacking DRAM vertically is constrained by chip pinout required to drive the added DRAM stacks, its wireability on top of silicon interposers, and thermal constraints
  - GPUs are close to the limits of processor die size so adding more 3D stack modules sacrifices budget for compute

# Memory Limits in Recommender Systems - Approach 1

*Model size is in the 100s of GB, much greater than GPU capacity*

- **CPU-only**: Run model on CPU. Gets around GPU memory bottleneck, but slow!

# Memory Limits in Recommender Systems - Approach 2

*Model size is in the 100s of GB, much greater than GPU capacity*

- **CPU-GPU hybrid**: CPU ‘memcpy‘ embeddings over to the GPU, then the GPU computes the output tensors and does the remainder of the recommender calculation. This bloats up memory bandwidth!

# Memory Limits in Recommender Systems - Figure

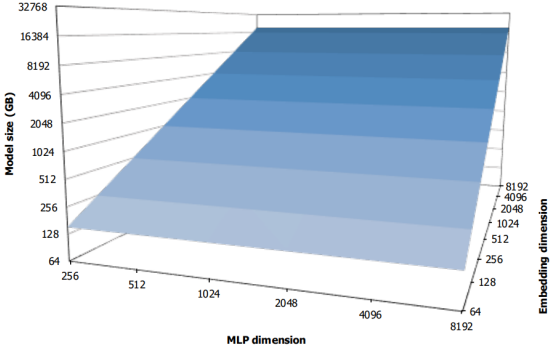



**Figure 3: Model size growth of neural collaborative filtering (NCF [26]) based recommender system when the MLP layer dimension size (x-axis) and the embedding vector dimension size (y-axis) is scaled up. Experiment assumes the embedding lookup table contains 5 million users and 5 million items per each lookup table. As shown, larger embeddings rather than larger MLP dimension size cause a much more dramatic increase in model size.**

# Towards a Scalable Memory System

*Main Challenge: Embeddings are in the embedding lookup table, which are stored in the capacity-optimized but bandwidth-limited CPU memory.*

- Prior solutions suffer from low-bandwidth embedding gather operations over CPU memory which adds significant latency

# Towards a Scalable Memory System

*Main Challenge: Embeddings are in the embedding lookup table, which are stored in the capacity-optimized but bandwidth-limited CPU memory.*

- Prior solutions suffer from low-bandwidth embedding gather operations over CPU memory which adds significant latency
- CPU-only and CPU-GPU recommender systems need to trade-off the computational bottleneck of low-throughput CPUs over the communication bottleneck of PCIe

# Towards a Scalable Memory System

*Main Challenge: Embeddings are in the embedding lookup table, which are stored in the capacity-optimized but bandwidth-limited CPU memory.*

- Prior solutions suffer from low-bandwidth embedding gather operations over CPU memory which adds significant latency
- CPU-only and CPU-GPU recommender systems need to trade-off the computational bottleneck of low-throughput CPUs over the communication bottleneck of PCIe
- **Urgent need for a system-level solution that provides scalable memory capacity and bandwidth expansion**

Section 3

# TensorDIMM

# Key Contributions

1. This work is the first to identify, analyze, and explore architectural solutions for “sparse” embedding layers.

# Key Contributions

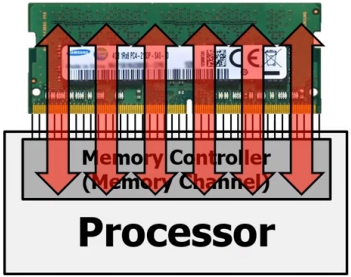
1. This work is the first to identify, analyze, and explore architectural solutions for "sparse" embedding layers.
2. **TensorDIMM**, a practical near-memory processing architecture built on top of commodity DRAMs which offers scalable increase in both memory capacity and bandwidth for embedding gathers and tensor operations.

# Key Contributions

1. This work is the first to identify, analyze, and explore architectural solutions for "sparse" embedding layers.
2. **TensorDIMM**, a practical near-memory processing architecture built on top of commodity DRAMs which offers scalable increase in both memory capacity and bandwidth for embedding gathers and tensor operations.
3. **TensorNode**, a TensorDIMM-based disaggregated memory system for DL. The efficiency of our solution is demonstrated as a proof-of-concept software prototype on a high-end GPU system, achieving significant performance improvements than conventional approaches.

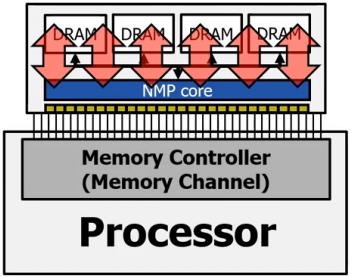
# Key Contributions

Embedding **gathers/reductions** are orchestrated between processor & DRAM




**Current system**

Embedding **gathers/reductions** are done **"locally"** within a DIMM




**TensorDIMM approach**

* Kwon et al., "TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning", MICRO-2019

# TensorDIMM Micro-Architecture

- Commodity buffered DIMMs enhanced with near memory processing (NMP) units placed *inside* the DIMM buffers

# TensorDIMM Micro-Architecture

- Commodity buffered DIMMs enhanced with near memory processing (NMP) units placed *inside* the DIMM buffers
  - **Vector ALU** customized for key DL tensor operations (gather, reduce, average)

# TensorDIMM Micro-Architecture

- Commodity buffered DIMMs enhanced with near memory processing (NMP) units placed *inside* the DIMM buffers
  - **Vector ALU** customized for key DL tensor operations (gather, reduce, average)
  - DDR interface

# TensorDIMM Micro-Architecture

- Commodity buffered DIMMs enhanced with near memory processing (NMP) units placed *inside* the DIMM buffers
  - **Vector ALU** customized for key DL tensor operations (gather, reduce, average)
  - DDR interface
  - Local memory controller with SRAM queues for buffering input / output tensors

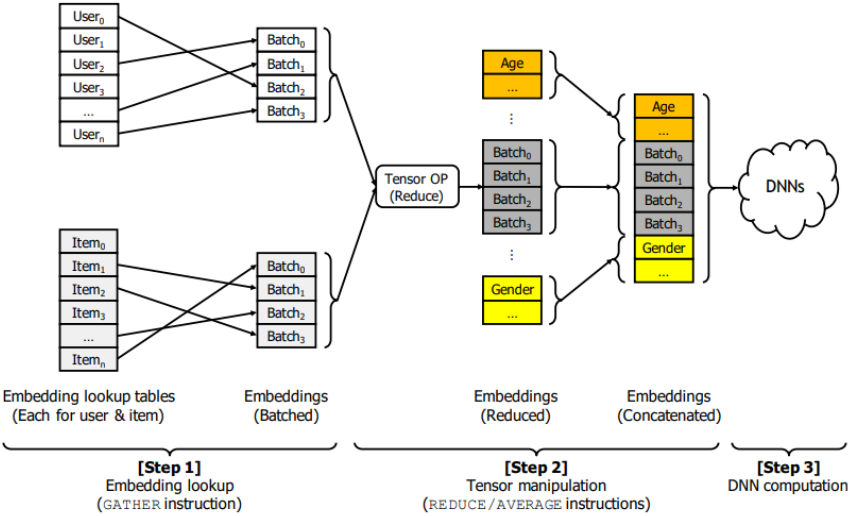
# TensorDIMM Micro-Architecture

- Commodity buffered DIMMs enhanced with near memory processing (NMP) units placed *inside* the DIMM buffers
  - **Vector ALU** customized for key DL tensor operations (gather, reduce, average)
  - DDR interface
  - Local memory controller with SRAM queues for buffering input / output tensors
- By performing gather + reduce/average operations in the DRAM, memory bandwidth requirements are significantly reduced

# TensorDIMM Micro-Architecture

- Commodity buffered DIMMs enhanced with near memory processing (NMP) units placed *inside* the DIMM buffers
  - **Vector ALU** customized for key DL tensor operations (gather, reduce, average)
  - DDR interface
  - Local memory controller with SRAM queues for buffering input / output tensors
- By performing gather + reduce/average operations in the DRAM, memory bandwidth requirements are significantly reduced
- Also functions as a normal DIMM if needed

# Vector ALU Functionality



# NMP Core Overhead

- Built on commercially available DRAMs
- Uses DDR PHY interface

# NMP Core Overhead

- Built on commercially available DRAMs
- Uses DDR PHY interface
- Instruction decode implemented as FSM inside memory controller

# NMP Core Overhead

- Built on commercially available DRAMs
- Uses DDR PHY interface
- Instruction decode implemented as FSM inside memory controller
- Major area + power overhead comes from SRAM queues

# NMP Core Overhead

- Built on commercially available DRAMs
- Uses DDR PHY interface
- Instruction decode implemented as FSM inside memory controller
- Major area + power overhead comes from SRAM queues
  - Must be large enough to hold bandwidth-delay product

# NMP Core Overhead

- Built on commercially available DRAMs
- Uses DDR PHY interface
- Instruction decode implemented as FSM inside memory controller
- Major area + power overhead comes from SRAM queues
  - Must be large enough to hold bandwidth-delay product
  - $\approx$ 0.5KB needed

# NMP Core Overhead

- Built on commercially available DRAMs
- Uses DDR PHY interface
- Instruction decode implemented as FSM inside memory controller
- Major area + power overhead comes from SRAM queues
  - Must be large enough to hold bandwidth-delay product
  - $\approx$ 0.5KB needed
- 150 MHZ clock

# NMP Core Overhead

- Built on commercially available DRAMs
- Uses DDR PHY interface
- Instruction decode implemented as FSM inside memory controller
- Major area + power overhead comes from SRAM queues
  - Must be large enough to hold bandwidth-delay product
  - $\approx$ 0.5KB needed
- 150 MHZ clock
- Negligable power + area overhead
  - IBM Centaur is a much more expensive design $\rightarrow$ TensorDIMM is commercially viable

# TensorNode

- Communication between DRAM and CPU memory occurs over slow PCIe connection

# TensorNode

- Communication between DRAM and CPU memory occurs over slow PCIe connection
  - Connecting TensorDIMM DRAMs over NVLINK interconnect is much faster (≈9x)

# TensorNode

- Communication between DRAM and CPU memory occurs over slow PCIe connection
  - Connecting TensorDIMM DRAMs over NVLINK interconnect is much faster (≈9x)
- Relies on purposeful address mapping to effectively utilize bandwidth

# TensorNode

- Communication between DRAM and CPU memory occurs over slow PCIe connection
  - Connecting TensorDIMM DRAMs over NVLINK interconnect is much faster (≈9x)
- Relies on purposeful address mapping to effectively utilize bandwidth
  - Consecutive 64B within an embedding vector are interleaved across different ranks

# TensorDIMM Architecture

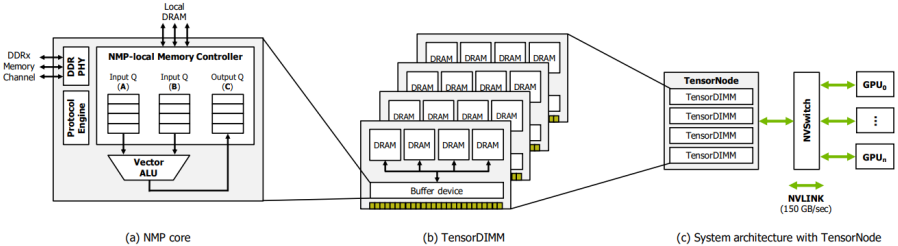
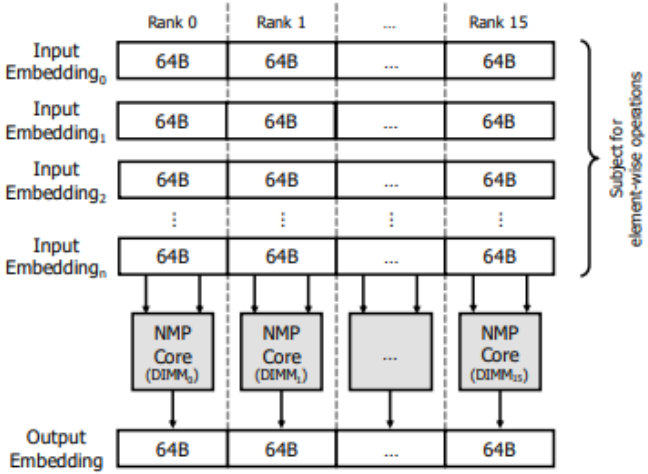



**Figure 6: High-level overview of our proposed system. (a) An NMP core is implemented inside the buffer device of each (b) `TensorDIMM`, multiples of which are employed as a (c) disaggregated memory pool, called `TensorNode`. The `TensorNode` is integrated inside the high-bandwidth GPU-side interconnect. The combination of NVLINK and NVSwitch enables GPUs to read/write data to/from `TensorNode` at a communication bandwidth up to 9× higher than PCIe.**

# Addressing Scheme



# Memory Bandwidth Scaling

- Conventional memory systems have a fixed maximum bandwidth per channel

# Memory Bandwidth Scaling

- Conventional memory systems have a fixed maximum bandwidth per channel
  - Physical memory channel bandwidth is multiplexed accross DIMMs/ranks

# Memory Bandwidth Scaling

- Conventional memory systems have a fixed maximum bandwidth per channel
  - Physical memory channel bandwidth is multiplexed accross DIMMs/ranks
- TensorNode introduces a disaggregated memory system

# Memory Bandwidth Scaling

- Conventional memory systems have a fixed maximum bandwidth per channel
  - Physical memory channel bandwidth is multiplexed accross DIMMs/ranks
- TensorNode introduces a disaggregated memory system
- Aggregate memory bandwidth scales with number of TensorDIMM units

# Memory Bandwidth Scaling

- Conventional memory systems have a fixed maximum bandwidth per channel
  - Physical memory channel bandwidth is multiplexed accross DIMMs/ranks
- TensorNode introduces a disaggregated memory system
- Aggregate memory bandwidth scales with number of TensorDIMM units
  - NMP cores access the TensorDIMM-internal DRAM chips “locally” within DIMMs (no sharing)

# Memory Bandwidth Scaling

- Conventional memory systems have a fixed maximum bandwidth per channel
  - Physical memory channel bandwidth is multiplexed accross DIMMs/ranks
- TensorNode introduces a disaggregated memory system
- Aggregate memory bandwidth scales with number of TensorDIMM units
  - NMP cores access the TensorDIMM-internal DRAM chips “locally” within DIMMs (no sharing)
  - Future-proof!

# TensorDIMM ISA and Runtime System

- ISA provides GPU with access to TensorDIMM functionalitites

# TensorDIMM ISA and Runtime System

- ISA provides GPU with access to TensorDIMM functionalitites
  - GATHER, REDUCE, AVERAGE

# TensorDIMM ISA and Runtime System

- ISA provides GPU with access to TensorDIMM functionalitites
  - GATHER, REDUCE, AVERAGE
  - These instructions are broadcast to **ALL** TensorDIMMs

# TensorDIMM ISA and Runtime System

- ISA provides GPU with access to TensorDIMM functionalitites
  - GATHER, REDUCE, AVERAGE
  - These instructions are broadcast to **ALL** TensorDIMMs
    - Each DIMM performs work on its local share of the data

# TensorDIMM ISA and Runtime System

- ISA provides GPU with access to TensorDIMM functionalitites
  - GATHER, REDUCE, AVERAGE
  - These instructions are broadcast to **ALL** TensorDIMMs
    - Each DIMM performs work on its local share of the data
- GPU runtime forwards CUDA kernel context to TensorNode network

Section 4

# Evaluation

# Evaluation Methodology

- Evaluated using hybrid methodology $\rightarrow$ cycle-level simulation and proof of concept prototype

# Evaluation Methodology

- Evaluated using hybrid methodology → cycle-level simulation and proof of concept prototype
- Cycle level simulation with Ramulator

# Evaluation Methodology

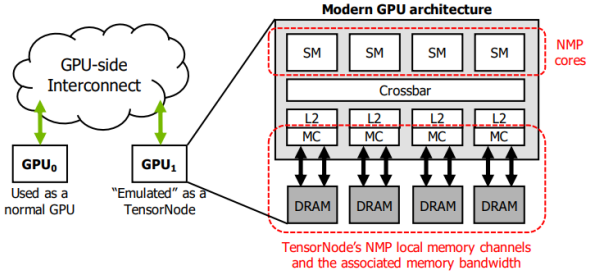
- Evaluated using hybrid methodology → cycle-level simulation and proof of concept prototype
- Cycle level simulation with Ramulator
  - Used to measure effective memory bandwidth utilization

# Evaluation Methodology

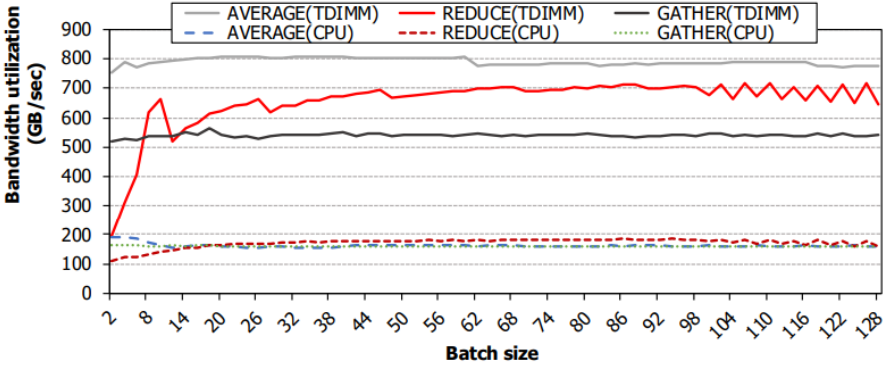
- Evaluated using hybrid methodology → cycle-level simulation and proof of concept prototype
- Cycle level simulation with Ramulator
  - Used to measure effective memory bandwidth utilization
- System behavior evaluated using dual-GPU setup

# Evaluation Methodology

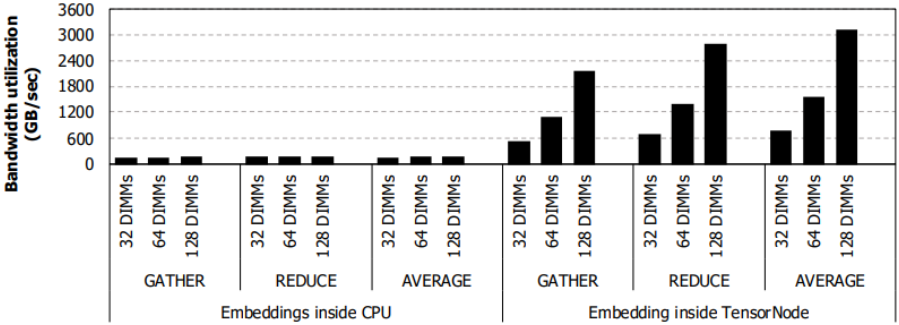
- Evaluated using hybrid methodology → cycle-level simulation and proof of concept prototype
- Cycle level simulation with Ramulator
  - Used to measure effective memory bandwidth utilization
- System behavior evaluated using dual-GPU setup



# Results


AVERAGE(TDIMM)
AVERAGE(CPU)
REDUCE(TDIMM)
REDUCE(CPU)
GATHER(TDIMM)
GATHER(CPU)
Bandwidth utilization (GB/sec)
900
800
700
600
500
400
300
200
100
0
2
8
14
20
26
32
38
44
50
56
62
68
74
80
86
92
98
104
110
116
122
128
Batch size

# Results


Bandwidth utilization (GB/sec)
3600
3000
2400
1800
1200
600
0
32 DIMMs
64 DIMMs
128 DIMMs
GATHER
REDUCE
AVERAGE
Embeddings inside CPU
Embedding inside TensorNode

# TLDR

- Memory bandwidth is the limiting factor for many DL workloads today

# TLDR

- Memory bandwidth is the limiting factor for many DL workloads today
- Computing near memory can reduce bandwidth usage / requirements for embedding operations

# TLDR

- Memory bandwidth is the limiting factor for many DL workloads today
- Computing near memory can reduce bandwidth usage / requirements for embedding operations
  - DL workloads are computationally repetitive $\rightarrow$ ideal candidates

# TLDR

- Memory bandwidth is the limiting factor for many DL workloads today
- Computing near memory can reduce bandwidth usage / requirements for embedding operations
  - DL workloads are computationally repetitive $\rightarrow$ ideal candidates
- **TensorDIMM makes embedding-focused DL workloads faster**

The end.